

Modeling Liquidity Spillover Dynamics

Adam Imran | 23i-5517 | BSFT 6th Semester, Spring 2026 AF3008 Business Research & Data Mining | Dr. Usama Arshad All figures saved at 300 DPI.

```
!pip install yfinance statsmodels scipy --quiet
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import adfuller
from scipy.optimize import minimize
import warnings
warnings.filterwarnings('ignore')

plt.rcParams['figure.dpi']      = 300
plt.rcParams['savefig.dpi']     = 300
plt.rcParams['font.family']     = 'serif'
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False
COLORS = ['#1f77b4', '#ff7f0e', '#d62728', '#9467bd', '#8c564b', '#2ca02c']
print('imports ok')
```

```
imports ok
```

1. Data Collection

```
tickers = {
    'SP500': '^GSPC',
    'NASDAQ': '^IXIC',
    'Bitcoin': 'BTC-USD',
    'Ethereum': 'ETH-USD',
    'Gold': 'GC=F',
    'Silver': 'SI=F'
}
START, END = '2020-01-01', '2024-12-31'

# download all tickers in one call - avoids MultiIndex issues
raw = yf.download(list(tickers.values()), start=START, end=END,
                  progress=False, auto_adjust=True)

ticker_to_name = {v: k for k, v in tickers.items()}

prices = raw['Close'].rename(columns=ticker_to_name)[list(tickers.keys())]
volumes = raw['Volume'].rename(columns=ticker_to_name)[list(tickers.keys())]

prices = prices.ffill().dropna(how='all')
volumes = volumes.ffill().dropna(how='all')

common = prices.index.intersection(volumes.index)
prices = prices.loc[common]
volumes = volumes.loc[common]

print(f'Assets : {list(prices.columns)}')
print(f'Rows : {len(prices)}')
print(f'Range : {prices.index[0].date()} to {prices.index[-1].date()}')
print(prices.tail(3))
```

```
Assets : ['SP500', 'NASDAQ', 'Bitcoin', 'Ethereum', 'Gold', 'Silver']
Rows : 1826
Range : 2020-01-01 to 2024-12-30
Ticker      SP500      NASDAQ      Bitcoin      Ethereum      Gold \
Date
2024-12-28  5970.839844  19722.029297  95163.929688  3397.902344  2617.199951
2024-12-29  5970.839844  19722.029297  93530.226562  3349.513428  2617.199951
2024-12-30  5906.939941  19486.789062  92643.210938  3356.392578  2606.100098
```

Ticker	Silver
Date	
2024-12-28	29.655001
2024-12-29	29.655001
2024-12-30	29.106001

2. Preprocessing & Amihud Illiquidity Ratio

```
log_ret = np.log(prices / prices.shift(1)).dropna()
ret = prices.pct_change().dropna()
dollar_vol = prices * volumes

amihud_raw = (log_ret.abs() / dollar_vol) * 1e8
amihud_raw = amihud_raw.replace([np.inf, -np.inf], np.nan)

idx = log_ret.index.intersection(amihud_raw.dropna(how='all').index)
log_ret = log_ret.loc[idx]
ret = ret.loc[idx]
amihud = amihud_raw.loc[idx].ffill().bfill().dropna()
log_ret = log_ret.loc[amihud.index]
ret = ret.loc[amihud.index]
prices_aln = prices.loc[amihud.index]

print(f'Clean observations: {len(amihud)}')
print('Amihud summary:')
print(amihud.describe().round(8).to_string())
```

Clean observations: 1824						
Amihud summary:						
Ticker	SP500	NASDAQ	Bitcoin	Ethereum	Gold	Silver
count	1.824000e+03	1.824000e+03	1.824000e+03	1.824000e+03	1824.000000	1824.000000
mean	3.000000e-08	1.000000e-08	0.000000e+00	2.600000e-07	4.725848	3776.854482
std	5.000000e-08	2.000000e-08	1.000000e-08	7.600000e-07	22.249321	13516.357198
min	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000	0.000000
25%	0.000000e+00	0.000000e+00	0.000000e+00	3.000000e-08	0.000000	0.000000
50%	2.000000e-08	1.000000e-08	0.000000e+00	8.000000e-08	0.400506	284.508377
75%	5.000000e-08	2.000000e-08	0.000000e+00	1.900000e-07	2.084444	2040.931775
max	6.900000e-07	4.100000e-07	1.700000e-07	2.215000e-05	519.140905	199684.554002

3. Descriptive Statistics

```
stats = pd.DataFrame({
    'Mean Ret (%)': (ret.mean()*100).round(4),
    'Std Dev (%)': (ret.std() *100).round(4),
    'Skewness': ret.skew().round(4),
    'Kurtosis': ret.kurt().round(4),
    'Ann. Vol (%)': (ret.std()*np.sqrt(252)*100).round(2),
    'Avg Amihud': amihud.mean().round(8)
})
print('Table 1: Descriptive Statistics (2020-2024)')
print(stats.to_string())
print()
print('Correlation Matrix:')
print(ret.corr().round(3).to_string())
```

Table 1: Descriptive Statistics (2020-2024)						
	Mean Ret (%)	Std Dev (%)	Skewness	Kurtosis	Ann. Vol (%)	Avg Amihud
Ticker						
SP500	0.0389	1.1161	-0.5717	20.3205	17.72	3.000000e-08
NASDAQ	0.0507	1.3321	-0.4673	10.3193	21.15	1.000000e-08
Bitcoin	0.1992	3.3555	-0.5212	11.1514	53.27	0.000000e+00
Ethereum	0.2754	4.3396	-0.3125	8.9186	68.89	2.600000e-07
Gold	0.0330	0.8506	-0.2286	6.5690	13.50	4.725848e+00
Silver	0.0422	1.7715	-0.2439	6.7397	28.12	3.776854e+03
Correlation Matrix:						
Ticker	SP500	NASDAQ	Bitcoin	Ethereum	Gold	Silver
Ticker						
SP500	1.000	0.946	0.354	0.350	0.128	0.207
NASDAQ	0.946	1.000	0.383	0.372	0.131	0.208
Bitcoin	0.354	0.383	1.000	0.817	0.118	0.158
Ethereum	0.350	0.372	0.817	1.000	0.116	0.156
Gold	0.128	0.131	0.118	0.116	1.000	0.780
Silver	0.207	0.208	0.158	0.156	0.780	1.000

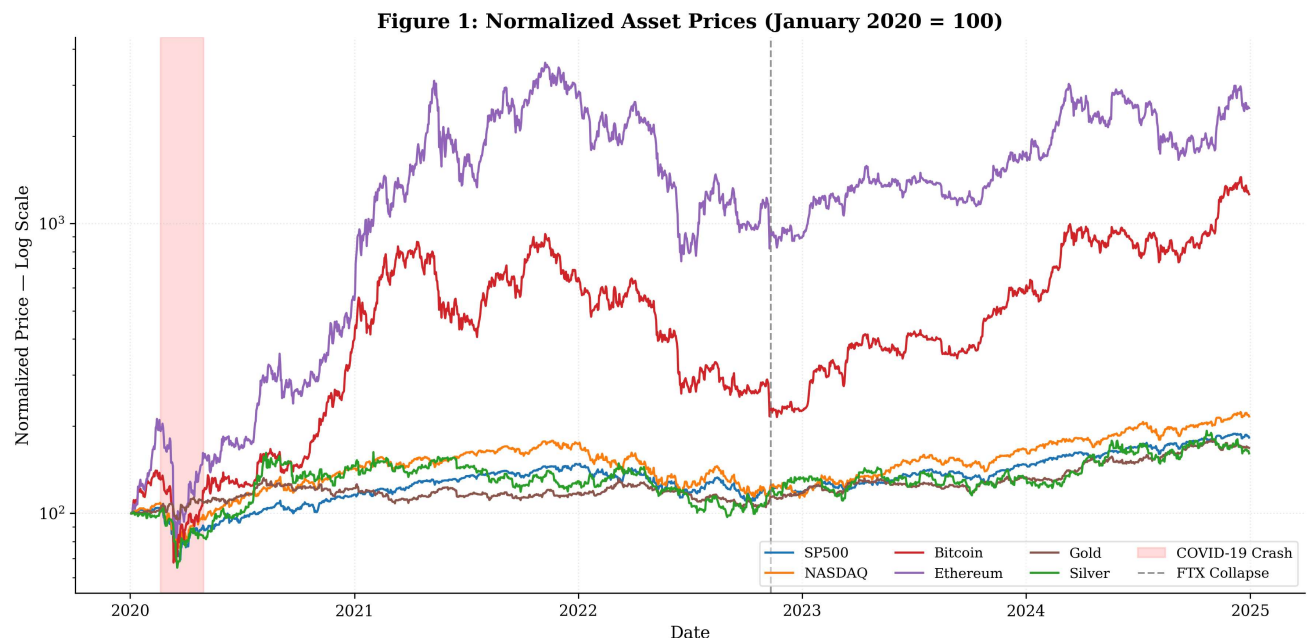
4. Export Dataset to CSV

```
p_exp = prices_aln.copy(); p_exp.columns = [f'Close_{c}' for c in p_exp.columns]
r_exp = rets.copy(); r_exp.columns = [f'Return_{c}' for c in r_exp.columns]
a_exp = amihud.copy(); a_exp.columns = [f'Amihud_{c}' for c in a_exp.columns]
dataset = pd.concat([p_exp, r_exp, a_exp], axis=1).dropna()
dataset.index.name = 'Date'
dataset.to_csv('dataset_liquidity_spillover.csv')
print(f'Exported: {dataset.shape[0]} rows x {dataset.shape[1]} cols')
```

Exported: 1824 rows x 18 cols

5. Figure 1: Normalized Prices

```
norm = prices_aln / prices_aln.iloc[0] * 100
fig, ax = plt.subplots(figsize=(12,6))
for i,col in enumerate(norm.columns):
    ax.plot(norm.index, norm[col], label=col, color=COLORS[i], linewidth=1.4)
ax.axvspan(pd.Timestamp('2020-02-20'), pd.Timestamp('2020-04-30'),
           alpha=0.12, color='red', label='COVID-19 Crash')
ax.axvline(pd.Timestamp('2022-11-11'), color='gray', linestyle='--',
           linewidth=1.2, alpha=0.8, label='FTX Collapse')
ax.set_title('Figure 1: Normalized Asset Prices (January 2020 = 100)',
            fontsize=13, fontweight='bold')
ax.set_xlabel('Date', fontsize=11)
ax.set_ylabel('Normalized Price - Log Scale', fontsize=11)
ax.legend(ncol=4, fontsize=9, framealpha=0.5)
ax.set_yscale('log')
ax.grid(True, alpha=0.22, linestyle=':')
plt.tight_layout()
plt.savefig('fig1_normalized_prices.png', dpi=300, bbox_inches='tight')
plt.show()
print('Saved fig1')
```



Saved fig1

6. Figure 2: Amihud Illiquidity Ratios

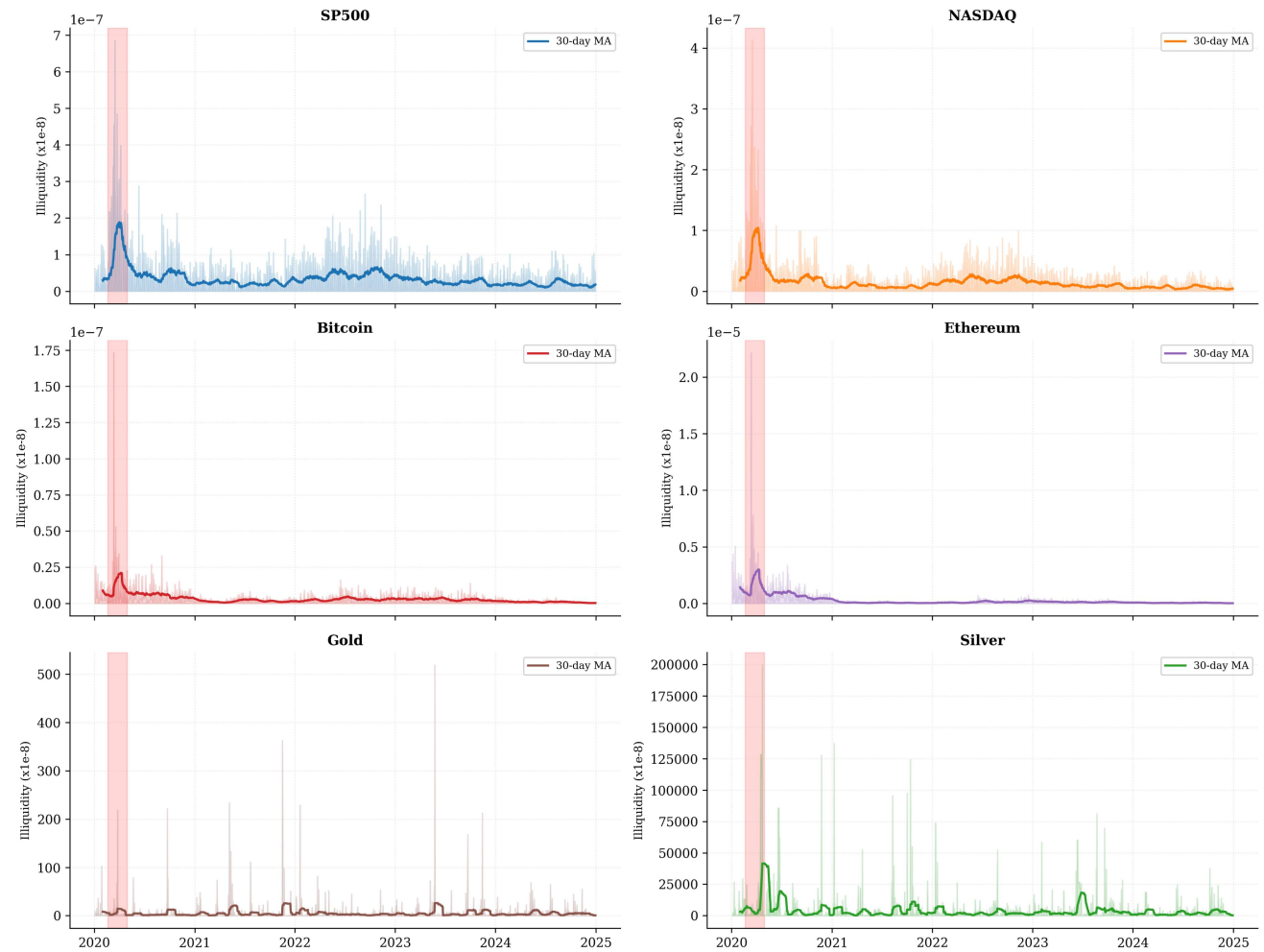
```
fig, axes = plt.subplots(3,2, figsize=(14,11), sharex=True)
axes = axes.flatten()
for i,col in enumerate(amihud.columns):
    ma30 = amihud[col].rolling(30).mean()
```

```

axes[i].fill_between(amihud.index, amihud[col], alpha=0.18, color=COLORS[i])
axes[i].plot(ma30.index, ma30, color=COLORS[i], linewidth=1.7, label='30-day MA')
axes[i].axvspan(pd.Timestamp('2020-02-20'), pd.Timestamp('2020-04-30'),
                alpha=0.14, color='red')
axes[i].set_title(col, fontsize=11, fontweight='bold')
axes[i].set_ylabel('Illiquidity (x1e-8)', fontsize=9)
axes[i].grid(True, alpha=0.2, linestyle=':')
axes[i].legend(fontsize=8)
fig.suptitle('Figure 2: Amihud Illiquidity Ratios (2020-2024) — Red = COVID Crash',
            fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('fig2_amihud_ratios.png', dpi=300, bbox_inches='tight')
plt.show()
print('Saved fig2')

```

Figure 2: Amihud Illiquidity Ratios (2020-2024) — Red = COVID Crash



Saved fig2

7. VAR Model & FEVD Spillover + Figure 3

```

amihud_log = np.log(amihud.clip(lower=1e-12)).replace([np.inf, -np.inf], np.nan).dropna()
amihud_diff = amihud_log.diff().dropna()

print('ADF on differenced log-Amihud:')
for col in amihud_diff.columns:
    stat, pval = adfuller(amihud_diff[col].dropna())[:2]
    print(f' {col:10s}: p={pval:.4f} -> {"STATIONARY" if pval<0.05 else "non-stationary"}')

var_model = VAR(amihud_diff)
p = max(var_model.select_order(maxlags=10).aic, 1)
print(f'VAR lag order (AIC): {p}')
var_res = var_model.fit(p)

```

```

ADF on differenced log-Amihud:
SP500      : p=0.0000 -> STATIONARY
NASDAQ     : p=0.0000 -> STATIONARY
Bitcoin    : p=0.0000 -> STATIONARY
Ethereum   : p=0.0000 -> STATIONARY
Gold       : p=0.0000 -> STATIONARY
Silver     : p=0.0000 -> STATIONARY
VAR lag order (AIC): 10

```

```

fevd = var_res.fevd(10)
assets = list(amihud.columns)
n = len(assets)
spill_mat = np.array([[fevd.decomp[i][-1][j]*100 for j in range(n)] for i in range(n)])

spill_df = pd.DataFrame(spill_mat, index=assets, columns=assets)
spill_df['From Others'] = 100 - np.diag(spill_mat)
spill_df.loc['To Others'] = pd.Series(
    [spill_mat[:,j].sum()-spill_mat[j,j] for j in range(n)] + [np.nan],
    index=assets+['From Others'])
tci = (spill_mat.sum()-np.trace(spill_mat))/(n*n)*100
print(f'Total Connectedness Index: {tci:.2f}%')
print(spill_df.round(2).to_string())

```

```

Total Connectedness Index: 740.91%

```

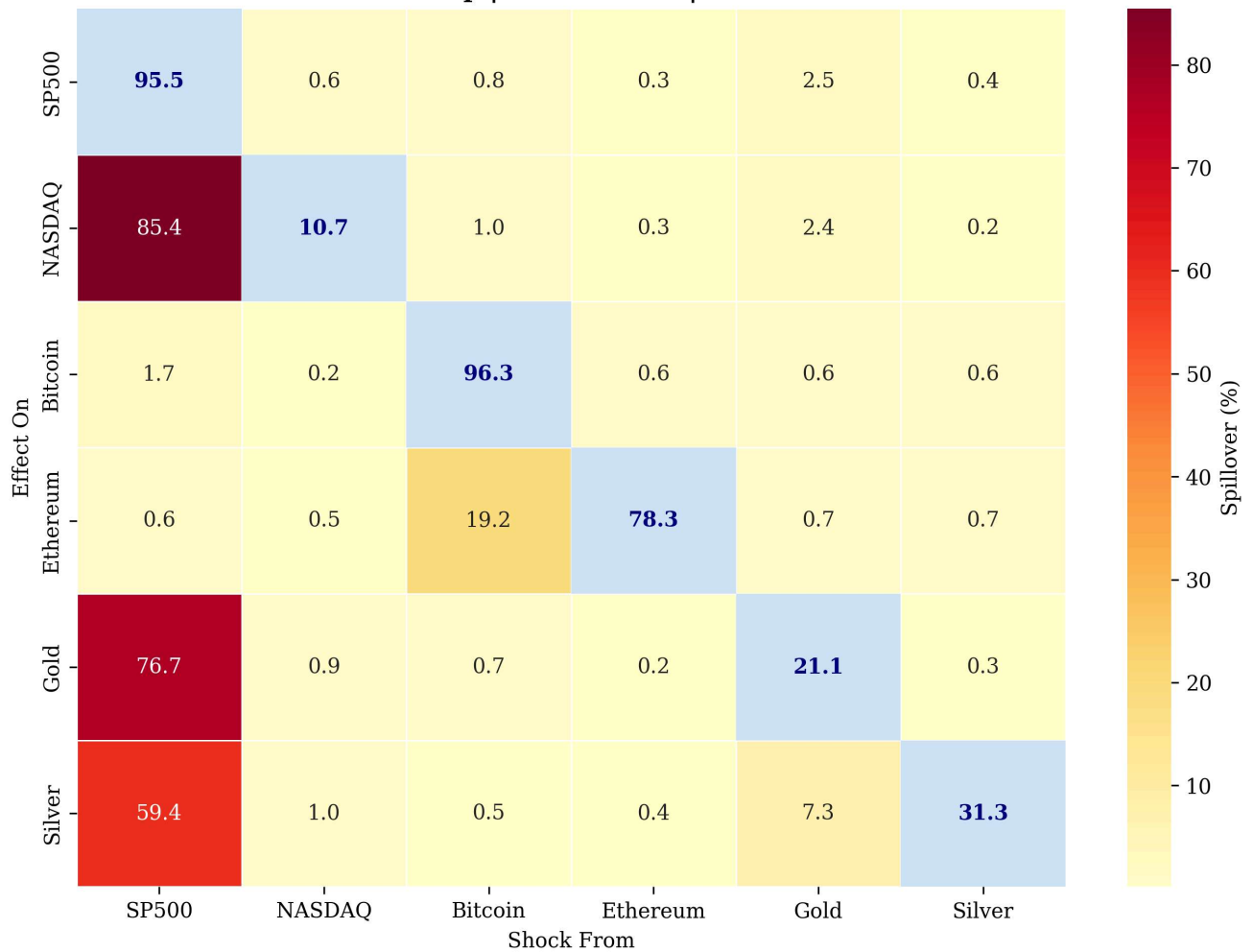
	SP500	NASDAQ	Bitcoin	Ethereum	Gold	Silver	From Others
SP500	95.47	0.58	0.80	0.27	2.51	0.38	4.53
NASDAQ	85.45	10.67	0.99	0.28	2.41	0.20	89.33
Bitcoin	1.73	0.18	96.33	0.61	0.56	0.59	3.67
Ethereum	0.56	0.50	19.17	78.33	0.74	0.70	21.67
Gold	76.70	0.90	0.75	0.21	21.13	0.31	78.87
Silver	59.39	1.02	0.48	0.45	7.33	31.33	68.67
To Others	223.82	3.18	22.19	1.82	13.54	2.18	NaN

```

heat = pd.DataFrame(spill_mat, index=assets, columns=assets)
fig, ax = plt.subplots(figsize=(9,7))
mask = np.eye(n, dtype=bool)
sns.heatmap(heat, annot=True, fmt='.1f', cmap='YlOrRd', linewidths=0.6,
            ax=ax, mask=mask, cbar_kws={'label':'Spillover (%)'}, annot_kws={'size':10})
for i in range(n):
    ax.add_patch(plt.Rectangle((i,i),1,1,fill=True,color='#cde5f7',lw=0))
    ax.text(i+0.5,i+0.5,f'{spill_mat[i,i]:.1f}',ha='center',va='center',
            fontsize=10,color='navy',fontweight='bold')
ax.set_title(f'Figure 3: Liquidity Spillover Connectedness\nFEVD 10-Step | TCI={tci:.1f}% | 2020-2024',
            fontsize=12, fontweight='bold')
ax.set_xlabel('Shock From', fontsize=10)
ax.set_ylabel('Effect On', fontsize=10)
plt.tight_layout()
plt.savefig('fig3_spillover_heatmap.png', dpi=300, bbox_inches='tight')
plt.show()
print('Saved fig3')

```

Figure 3: Liquidity Spillover Connectedness
FEVD 10-Step | TCI=740.9% | 2020-2024



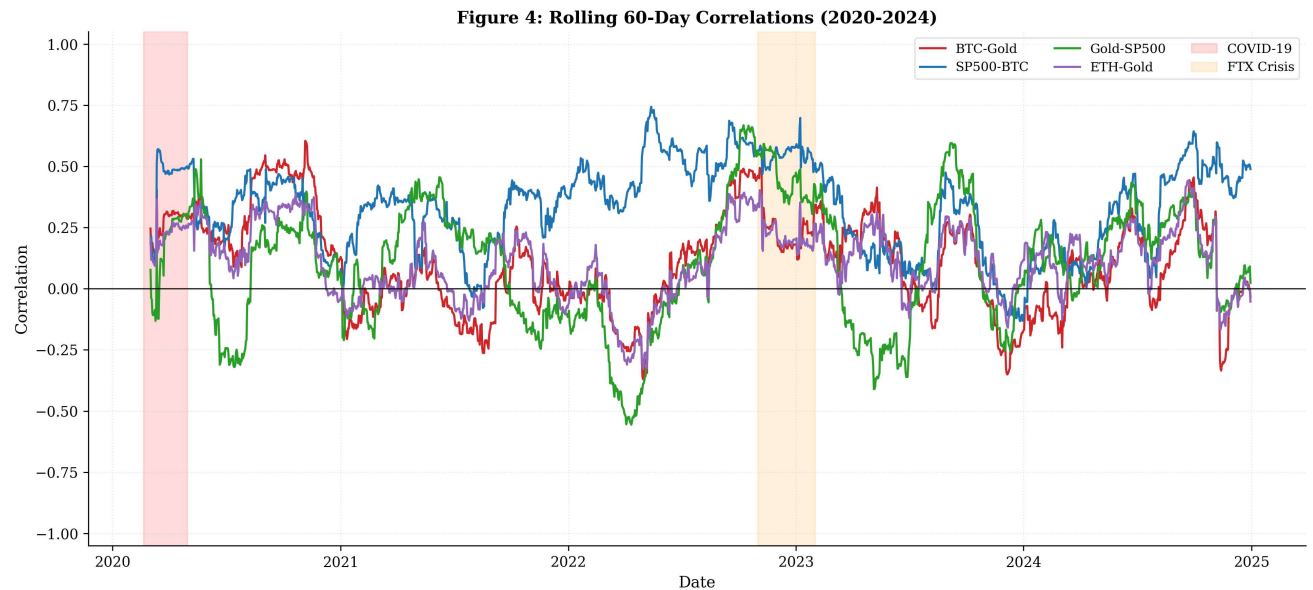
Saved fig3

8. Figure 4: Rolling Correlations

```

pairs = [('Bitcoin','Gold','#d62728','BTC-Gold'),
         ('SP500','Bitcoin','#1f77b4','SP500-BTC'),
         ('Gold','SP500','#2ca02c','Gold-SP500'),
         ('Ethereum','Gold','#9467bd','ETH-Gold')]
fig, ax = plt.subplots(figsize=(13,6))
for a,b,c,lbl in pairs:
    rc = ret[a].rolling(60).corr(ret[b])
    ax.plot(rc.index, rc, label=lbl, color=c, linewidth=1.5)
ax.axhline(0, color='black', linewidth=0.8)
ax.axvspan(pd.Timestamp('2020-02-20'),pd.Timestamp('2020-04-30'),
            alpha=0.12,color='red',label='COVID-19')
ax.axvspan(pd.Timestamp('2022-11-01'),pd.Timestamp('2023-02-01'),
            alpha=0.12,color='orange',label='FTX Crisis')
ax.set_title('Figure 4: Rolling 60-Day Correlations (2020-2024)',fontsize=12,fontweight='bold')
ax.set_xlabel('Date',fontsize=11); ax.set_ylabel('Correlation',fontsize=11)
ax.legend(fontsize=9,ncol=3,framealpha=0.5); ax.set_ylim(-1.05,1.05)
ax.grid(True,alpha=0.22,linestyle=':')
plt.tight_layout()
plt.savefig('fig4_rolling_correlations.png',dpi=300, bbox_inches='tight')
plt.show(); print('Saved fig4')

```



Saved fig4

9. Figure 5: Regime Correlations

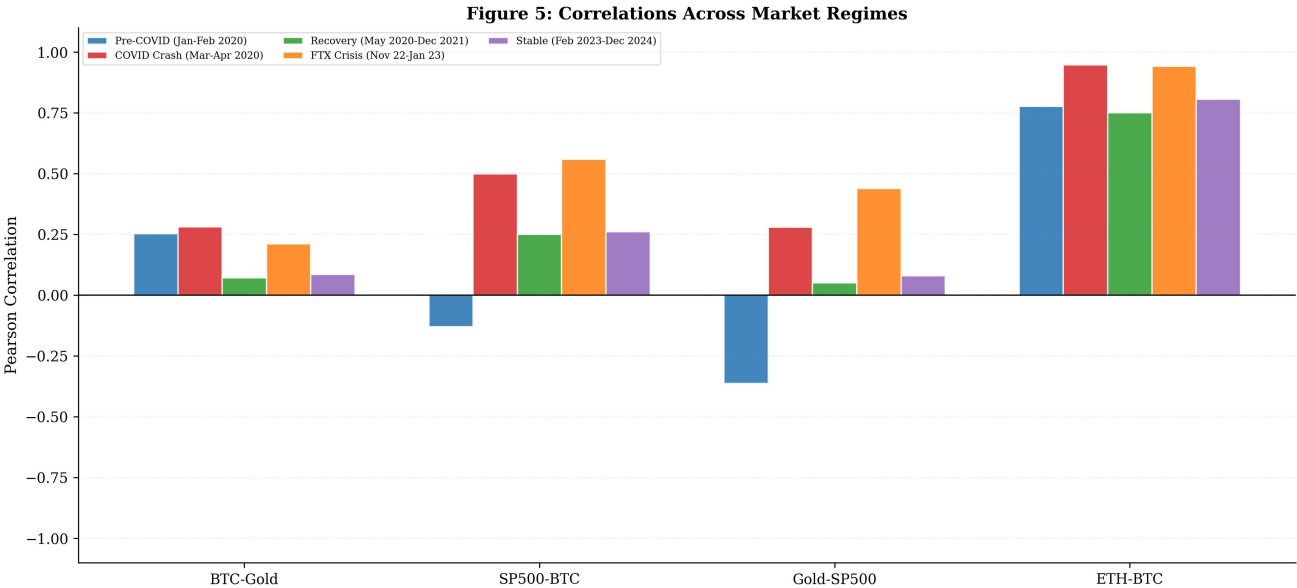
```

periods = {
    'Pre-COVID (Jan-Feb 2020)': ('2020-01-01', '2020-02-19'),
    'COVID Crash (Mar-Apr 2020)': ('2020-02-20', '2020-04-30'),
    'Recovery (May 2020-Dec 2021)': ('2020-05-01', '2021-12-31'),
    'FTX Crisis (Nov 22-Jan 23)': ('2022-11-01', '2023-01-31'),
    'Stable (Feb 2023-Dec 2024)': ('2023-02-01', '2024-12-31')
}

kp = [('Bitcoin', 'Gold'), ('SP500', 'Bitcoin'), ('Gold', 'SP500'), ('Ethereum', 'Bitcoin')]
pl = ['BTC-Gold', 'SP500-BTC', 'Gold-SP500', 'ETH-BTC']
cd = {}
for nm, (s, e) in periods.items():
    sub = ret.loc[s:e]
    if len(sub) >= 10: cd[nm] = [sub[a].corr(sub[b]) for a, b in kp]
corr_df = pd.DataFrame(cd, index=pl)

fig, ax = plt.subplots(figsize=(13, 6))
x = np.arange(len(pl)); w = 0.15
pcols = ['#1f77b4', '#d62728', '#2ca02c', '#ff7f0e', '#9467bd']
for k, (period, col) in enumerate(zip(corr_df.columns, pcols)):
    ax.bar(x+k*w, corr_df[period], w, label=period, color=col, alpha=0.85, edgecolor='white')
ax.axhline(0, color='black', linewidth=0.9)
ax.set_xticks(x+w*2); ax.set_xticklabels(pl, fontsize=10)
ax.set_title('Figure 5: Correlations Across Market Regimes', fontsize=12, fontweight='bold')
ax.set_ylabel('Pearson Correlation', fontsize=11)
ax.legend(fontsize=7, ncol=3, framealpha=0.5); ax.set_ylim(-1.1, 1.1)
ax.grid(True, alpha=0.22, axis='y', linestyle=':')
plt.tight_layout()
plt.savefig('fig5_regime_correlations.png', dpi=300, bbox_inches='tight')
plt.show(); print('Saved fig5')
print(corr_df.round(3).to_string())

```



Saved fig5

	Pre-COVID (Jan-Feb 2020)	COVID Crash (Mar-Apr 2020)	Recovery (May 2020-Dec 2021)	FTX Crisis (Nov 22-Jan 23)	Stable (Feb 2023-Dec 2024)
BTC-Gold	0.252	0.280	0.071	0.211	0.211
SP500-BTC	-0.130	0.499	0.250	0.559	0.559
Gold-SP500	-0.363	0.279	0.050	0.439	0.439
ETH-BTC	0.776	0.947	0.750	0.940	0.940

10. Figure 6: Portfolio Optimization

```
mu = ret.mean()*252
cov = ret.cov()*252
liq_norm = amihud.mean()/amihud.mean().sum()
n_a=len(mu); w0=np.ones(n_a)/n_a
bnds=[(0,0.60)]*n_a
cons=[{'type':'eq','fun':lambda w:w.sum()-1}]

def pret(w): return w@mu.values
def pvol(w): return np.sqrt(w@cov.values@w)
def neg_sharpe(w): return -pret(w)/pvol(w)
def neg_sharpe_liq(w,lam=0.4): return -pret(w)/pvol(w)+lam*(w@liq_norm.values)

res_std=minimize(neg_sharpe, w0,method='SLSQP',bounds=bnds,constraints=cons)
res_liq=minimize(neg_sharpe_liq,w0,method='SLSQP',bounds=bnds,constraints=cons)
w_std=res_std.x; w_liq=res_liq.x

print('Standard MV:')
for nm,w in zip(ret.columns,w_std): print(f' {nm:10s}: {w*100:.1f}%')
print(f' Sharpe={-neg_sharpe(w_std):.4f}')
print('Liquidity-Adjusted:')
for nm,w in zip(ret.columns,w_liq): print(f' {nm:10s}: {w*100:.1f}%')
print(f' Sharpe={-neg_sharpe(w_liq):.4f}')
```

Standard MV:
SP500 : 0.0%
NASDAQ : 15.3%
Bitcoin : 8.8%
Ethereum : 15.9%
Gold : 60.0%
Silver : 0.0%
Sharpe=1.1546
Liquidity-Adjusted:
SP500 : 0.0%
NASDAQ : 15.3%
Bitcoin : 8.8%
Ethereum : 15.9%
Gold : 60.0%
Silver : 0.0%
Sharpe=1.1546

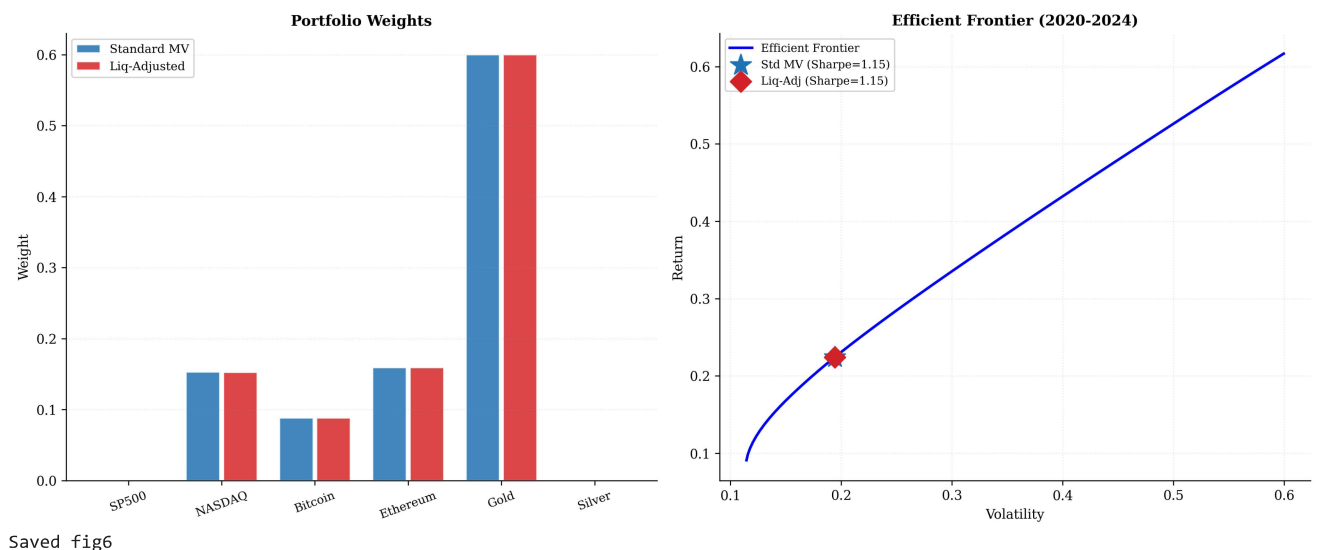

```

ef_ret,ef_vol=[],[]
for target in np.linspace(mu.min(),mu.max(),80):
    c=cons+[{ 'type':'eq', 'fun':lambda w,t=target:pret(w)-t}]
    r=minimize(pvol,w0,method='SLSQP',bounds=bnds,constraints=c)
    if r.success: ef_ret.append(pret(r.x)); ef_vol.append(pvol(r.x))

fig,(ax1,ax2)=plt.subplots(1,2,figsize=(14,6))
x=np.arange(n_a)
ax1.bar(x-0.2,w_std,0.36,label='Standard MV',color='#1f77b4',alpha=0.85,edgecolor='white')
ax1.bar(x+0.2,w_liq,0.36,label='Liq-Adjusted',color='#d62728',alpha=0.85,edgecolor='white')
ax1.set_xticks(x); ax1.set_xticklabels(list(ret.columns),rotation=20,fontsize=9)
ax1.set_title('Portfolio Weights',fontsize=11,fontweight='bold')
ax1.set_ylabel('Weight'); ax1.legend(fontsize=9)
ax1.grid(True,alpha=0.22,axis='y',linestyle=':')

ax2.plot(ef_vol,ef_ret,'b-',linewidth=2,label='Efficient Frontier')
ax2.scatter(pvol(w_std),pret(w_std),marker='*',s=280,color='#1f77b4',zorder=5,
            label=f'Std MV (Sharpe={-neg_sharpe(w_std):.2f})')
ax2.scatter(pvol(w_liq),pret(w_liq),marker='D',s=130,color='#d62728',zorder=5,
            label=f'Liq-Adj (Sharpe={-neg_sharpe(w_liq):.2f})')
ax2.set_xlabel('Volatility',fontsize=10); ax2.set_ylabel('Return',fontsize=10)
ax2.set_title('Efficient Frontier (2020-2024)',fontsize=11,fontweight='bold')
ax2.legend(fontsize=8.5); ax2.grid(True,alpha=0.22,linestyle=':')
fig.suptitle('Figure 6: Portfolio Optimization',fontsize=13,fontweight='bold',y=1.01)
plt.tight_layout()
plt.savefig('fig6_portfolio_optimization.png',dpi=300,bbox_inches='tight')
plt.show(); print('Saved fig6')

```

Figure 6: Portfolio Optimization

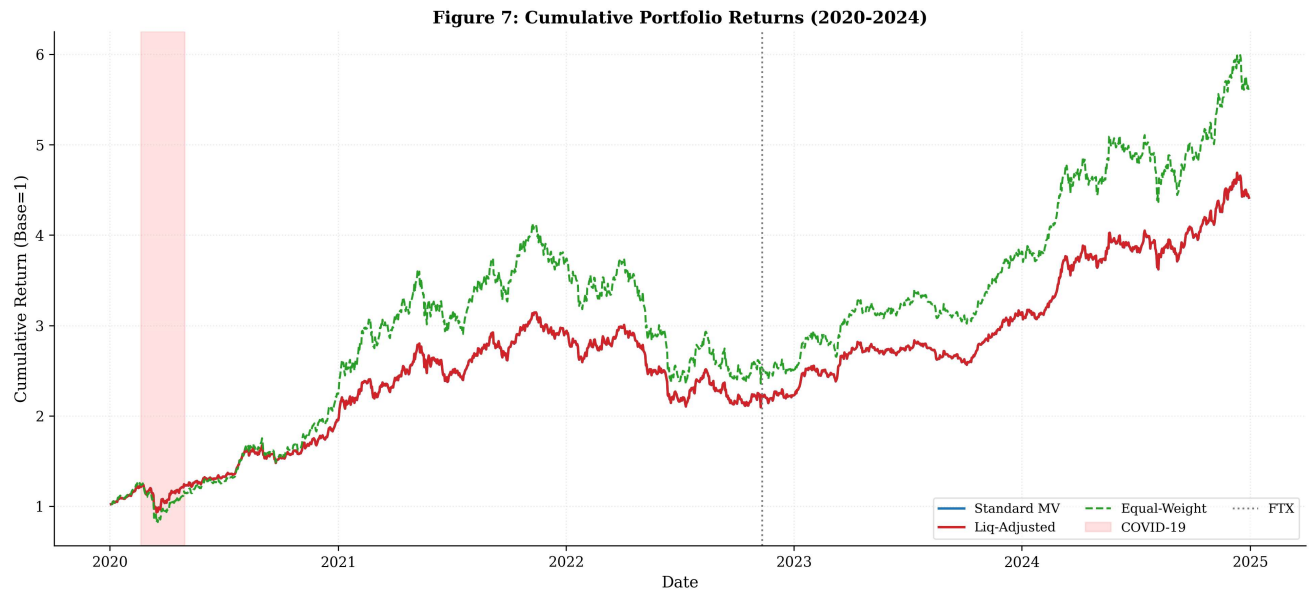
11. Figure 7: Cumulative Returns

```

w_eq=np.ones(n_a)/n_a
cum_std=(1+ret@w_std).cumprod()
cum_liq=(1+ret@w_liq).cumprod()
cum_eq =(1+ret@w_eq ).cumprod()
fig,ax=plt.subplots(figsize=(13,6))
ax.plot(cum_std.index,cum_std,label='Standard MV',color='#1f77b4',linewidth=1.7)
ax.plot(cum_liq.index,cum_liq,label='Liq-Adjusted',color='#d62728',linewidth=1.7)
ax.plot(cum_eq.index, cum_eq, label='Equal-Weight',color='#2ca02c',linewidth=1.4,linestyle='--')
ax.axvspan(pd.Timestamp('2020-02-20'),pd.Timestamp('2020-04-30'),
            alpha=0.10,color='red',label='COVID-19')
ax.axvline(pd.Timestamp('2022-11-11'),color='gray',linestyle=':',linewidth=1.3,label='FTX')
ax.set_title('Figure 7: Cumulative Portfolio Returns (2020-2024)',fontsize=12,fontweight='bold')
ax.set_xlabel('Date',fontsize=11); ax.set_ylabel('Cumulative Return (Base=1)',fontsize=11)
ax.legend(fontsize=9,ncol=3,framealpha=0.5)
ax.grid(True,alpha=0.22,linestyle=':')
plt.tight_layout()
plt.savefig('fig7_cumulative_returns.png',dpi=300,bbox_inches='tight')
plt.show()

```

```
print(f'Final: Std={cum_std.iloc[-1]:.2f}x  Liq={cum_liq.iloc[-1]:.2f}x  EW={cum_eq.iloc[-1]:.2f}x')
print('Saved fig7')
```



12. Figure 8: Safe-Haven Test

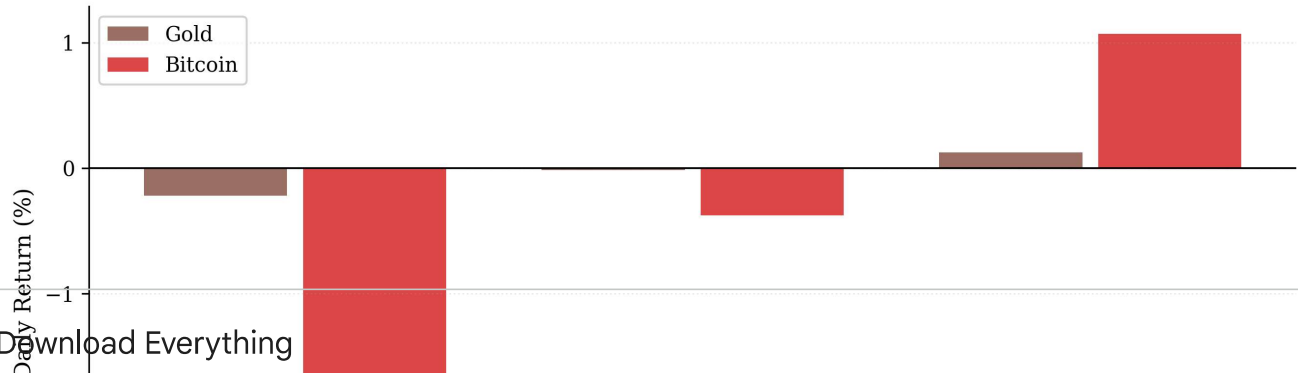
```
sp_r=ret['SP500']
q5,q25=sp_r.quantile(0.05),sp_r.quantile(0.25)
dn_ex=ret[sp_r<=q5]; dn_ml=ret[(sp_r>q5)&(sp_r<=q25)]; up=ret[sp_r>q25]
sh=pd.DataFrame({
    'Extreme Down (5%)': [dn_ex['Gold'].mean(), dn_ex['Bitcoin'].mean()],
    'Mild Down (5-25%)': [dn_ml['Gold'].mean(), dn_ml['Bitcoin'].mean()],
    'Positive Days': [up['Gold'].mean(), up['Bitcoin'].mean()]
},index=['Gold','Bitcoin'])
print('Safe-Haven Test [% returns]:')
print((sh*100).round(4).to_string())

fig,ax=plt.subplots(figsize=(9,5))
x=np.arange(3)
ax.bar(x-0.2,sh.loc['Gold']*100,0.36,label='Gold', color='#8c564b',alpha=0.85)
ax.bar(x+0.2,sh.loc['Bitcoin']*100,0.36,label='Bitcoin',color='#d62728',alpha=0.85)
ax.axhline(0,color='black',linewidth=0.9)
ax.set_xticks(x); ax.set_xticklabels(sh.columns,fontsize=9.5)
ax.set_ylabel('Avg Daily Return (%)',fontsize=11)
ax.set_title('Figure 8: Safe-Haven Test – Gold vs Bitcoin\nConditional on S&P 500',
            fontsize=11,fontweight='bold')
ax.legend(fontsize=10); ax.grid(True,alpha=0.22,axis='y',linestyle=':')
plt.tight_layout()
plt.savefig('fig8_safe_haven.png',dpi=300,bbox_inches='tight')
plt.show(); print('Saved fig8')
```

Safe-Haven Test [% returns]:

	Extreme Down (5%)	Mild Down (5-25%)	Positive Days
Gold	-0.2172	-0.0149	0.1257
Bitcoin	-3.3007	-0.3747	1.0711

Figure 8: Safe-Haven Test — Gold vs Bitcoin
Conditional on S&P 500



13. Download Everything